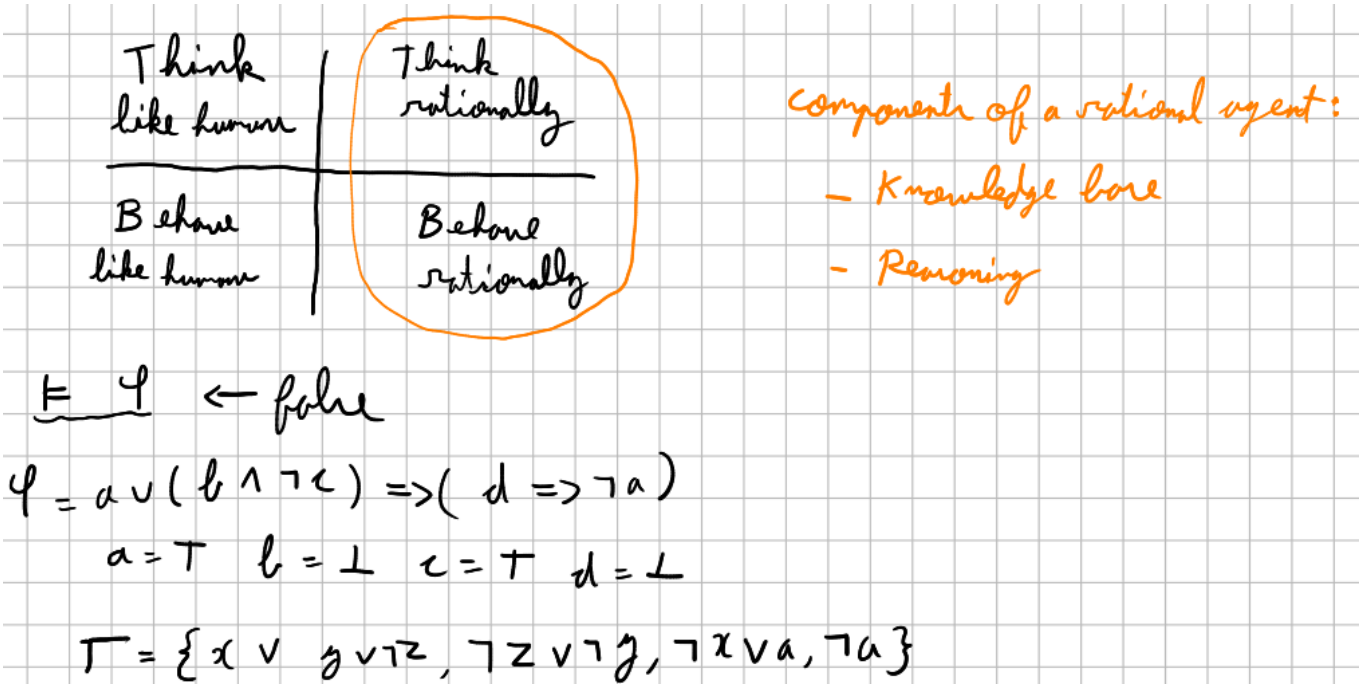
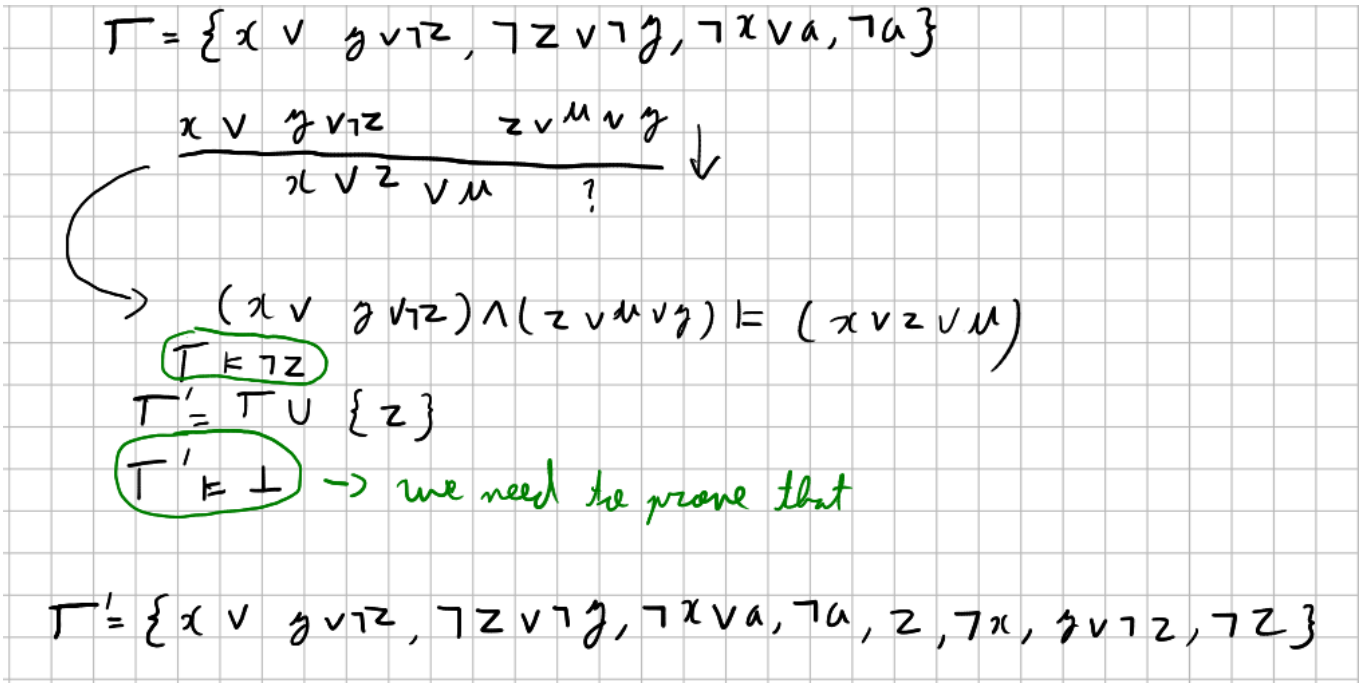
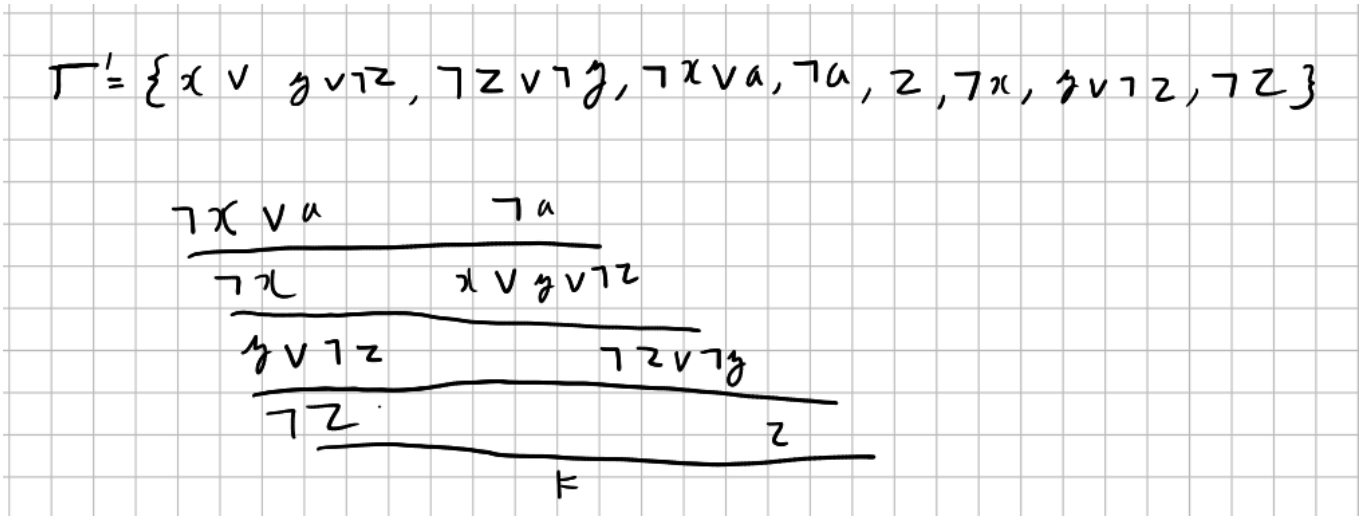
Revision/Recap

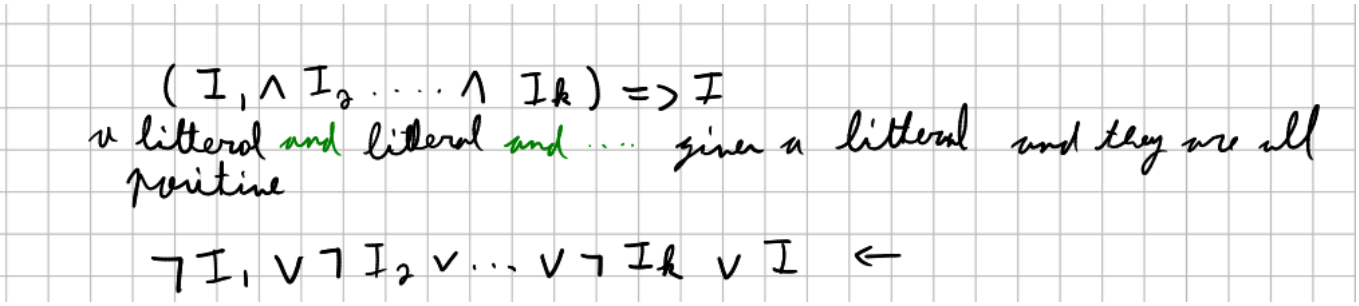




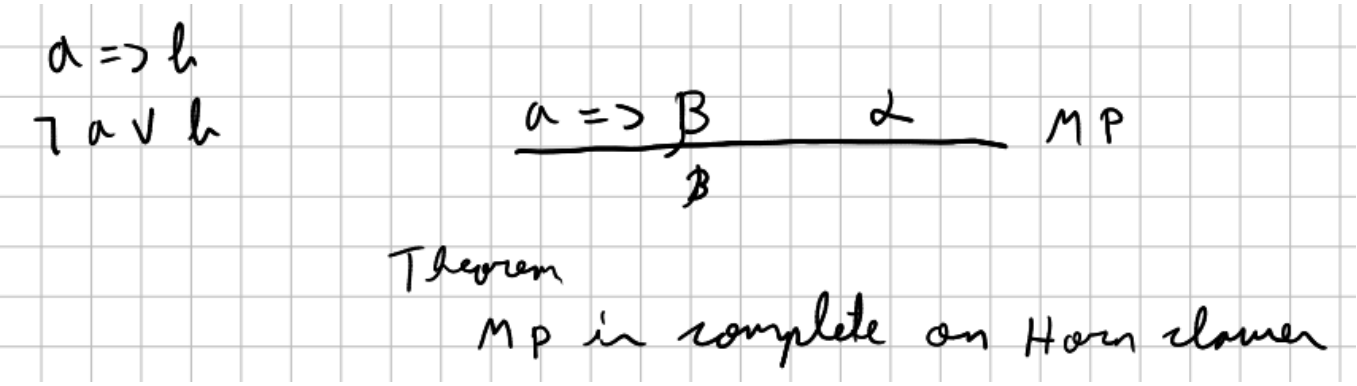


Lecture

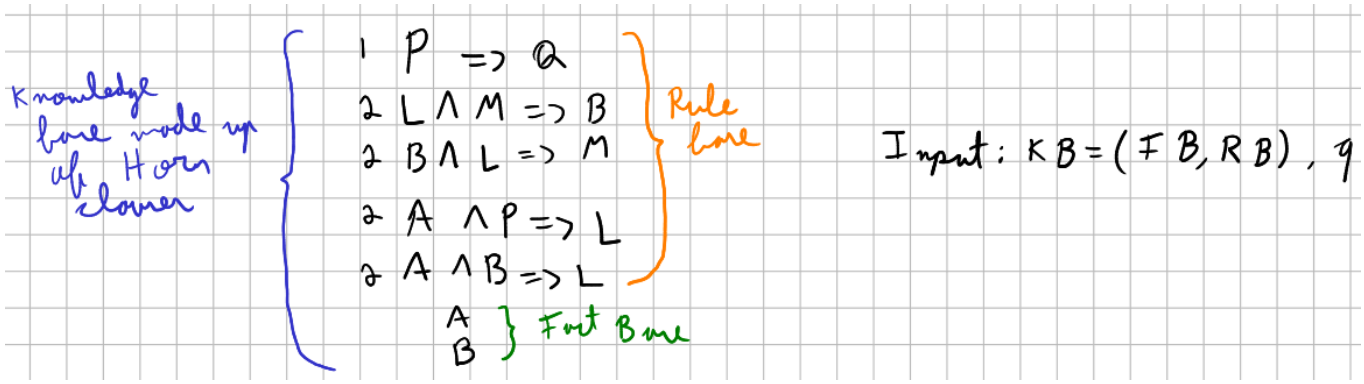
Horn clause

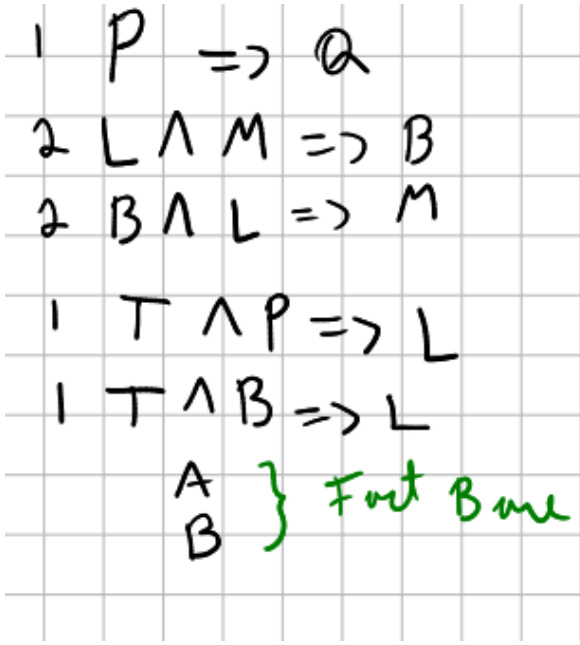


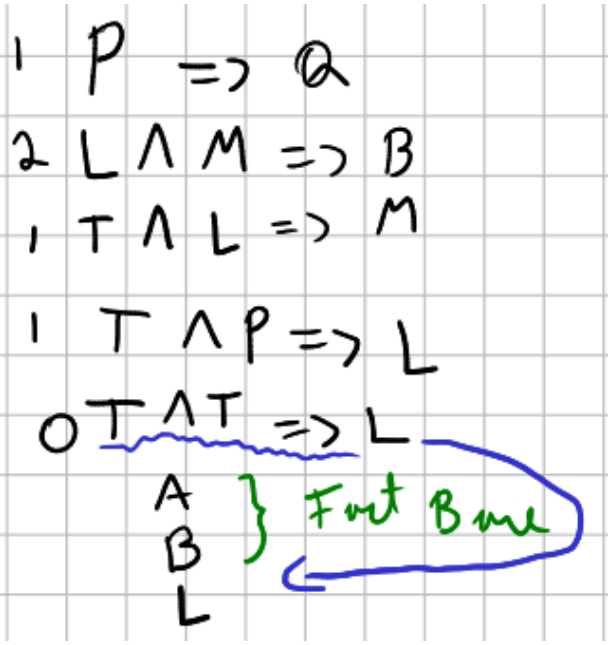
If my formula is made up of only horn clauses then we can solve the equation in polynomial time => it will be much faster

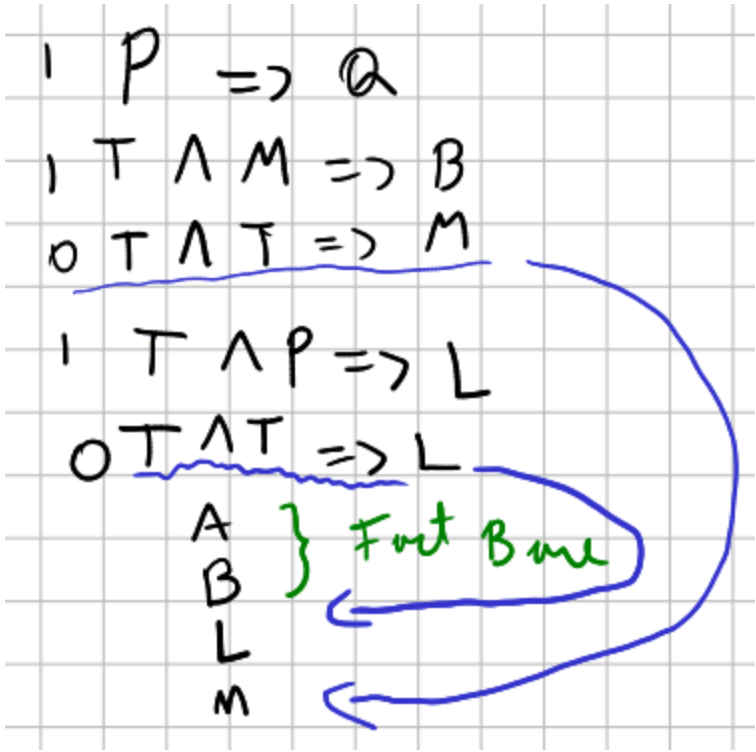


Forward Chaining algorithm

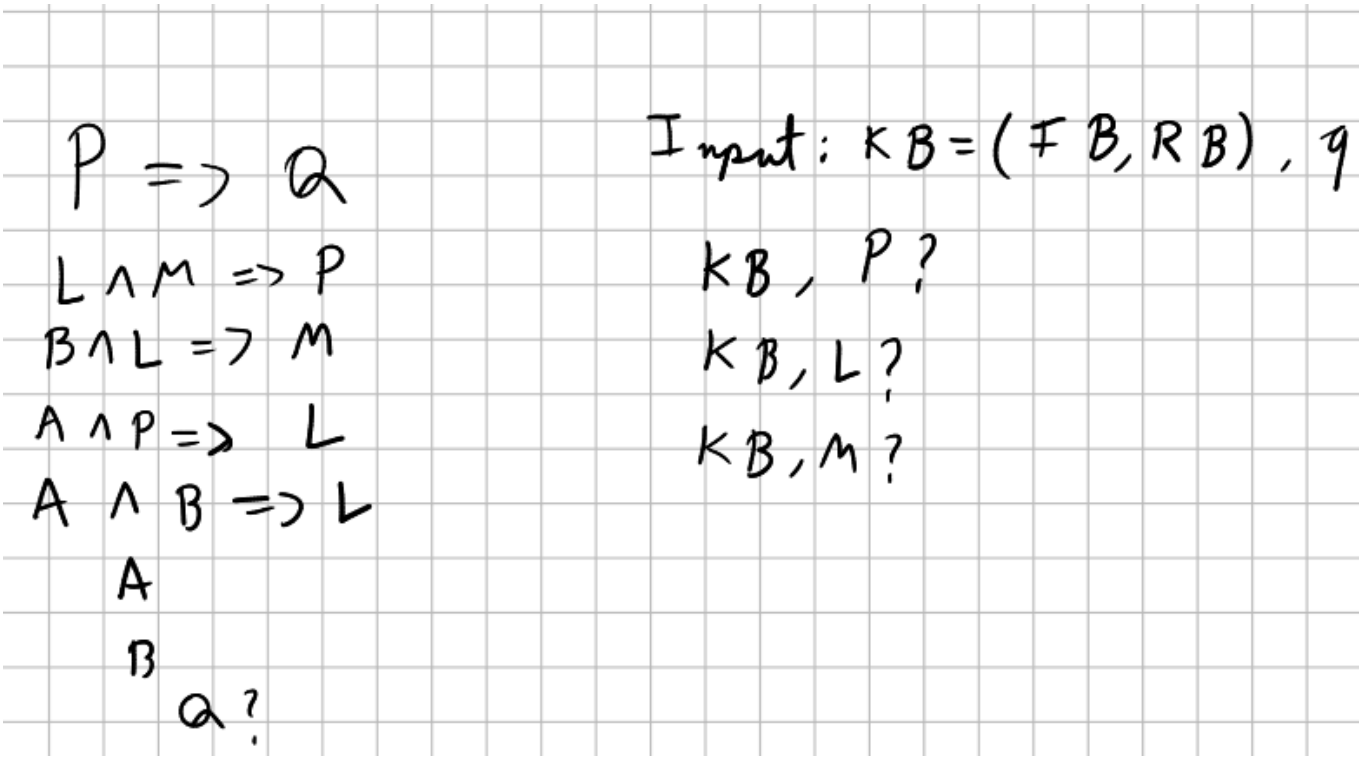






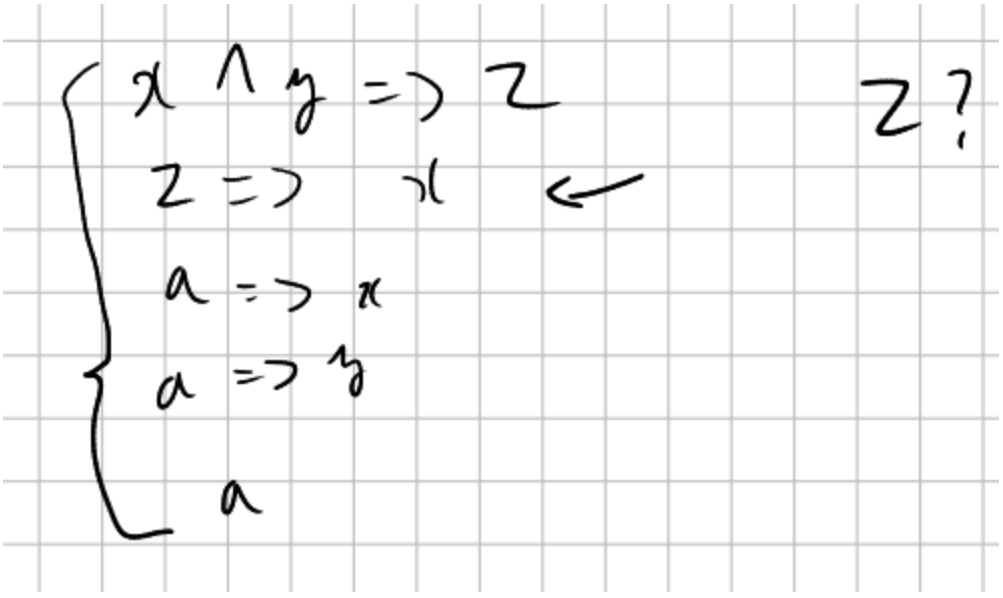


Backward Chaining algorithm



In Forward chaining we calculate all the statements and we deduce everything to be deduced.

With backward chaining, due to its recursive nature, we only calculate what needs to be calculated. We ignore redundant statements. In this example Forward and Backward chaining result in the same calculations.



In this example we will loop forever since if we want to evaluate `z` we need to evaluate `x`, but `x` relies on `z` so it is gonna recurse forever.

To solve this problem we can create a global array that keeps track of variables that we already evaluated:

```
active = { }
function Back...
    if a ∈ active
        return false
    if a ∈ I.B
        return T
    active = active ∪ {q}
    for c = (I1 ∧ I2 ∧ ... In) => q:
        k ← n
        b ← true
        while b ∧ k > 0
            b ∈ Backward(KB, Ik)
        if k == 0 ∧ b:
            active = active \ {q}
            return true
```

$\exists x \quad Q(x)$

$\forall x \quad P(x)$

$\exists n \quad \forall p \qquad p \times n = 0$

$$\exists x P(x) \equiv \bigvee_{x \in D} P(x)$$

$$\forall x P(x) \equiv \bigwedge_{x \in D} P(x)$$

$\exists x (P(x) \vee Q(x))$

$\exists \exists \exists P \Rightarrow (x)$ ✗